There are 5 major aspects of the networking portion of the code

- Connecting to the server to Login/create a new profile
- Sending the Leaderboard information from the server to client
- Chatting between clients during match's
- Updating the board state after the opponent moves
- And Last but not least the dedicated server that acts as a relay between the two clients

Connecting to the server to Login/create a new profile & Sending the Leaderboard information from the server to client

The first two are mostly covered by the Authentication & Profile Team as well as the Leaderboard & Matchmaking Team, so we simply will need to utilize the functions they end up making and sending the data they prepare over our connection

For our part of authentication: we should have a function that is called by the auth team, that takes in the username and password and preps them for being sent to the server, where the server compares them against its own file, and then another function to tell the auth teams code if the checks passed or fail, this function would be called by our the clients-side loop after it collects the servers response

Then there should also be a function for creating a new user, as well as another function to grab the data the server returns and let the auth team's code know it if succeeded and to proceed or if that user already exists or something like that

The auth team should be responsible for the code that compares username and password, we just need our server code to call it and return the results

For the leaderboard we should instead have the server regularly send over the stored leaderboard data, so we just need to have a function that decodes that data and updates the leaderboard code

Ill try to talk to the leaderboard team to see what kind of data structures they will be using to store this information, until then we can leave this on the backburner

Once we make the server-side portion of the code we will need another function to encode this data as well as read and write it to a file so it persists over server restarts

Chatting between clients during match's

For chatting between clients, we will want a message class that holds information such as “Message contents”, “Time”, “Username of sender” things like this. The GUI team will be responsible for collecting the users message and calling one our SendMessage(Message msg) function and passing a message object to us. From there we will need to format this information into a byte pattern which we can decode later and place this information into a local variable within the Networking class called something like “msgToBeSent” for the data to be sent by the connection when it is ready(Multithreading means we can’t force things to happen when we want to, but instead we wait until its ready).

We will also need a sister function to this SendMessage(Bytes someName) function, that does the opposite, it would take the bytes received from the server and translate them back into a Message() object, after that it would need to push it to the GUI team somehow to display it. Before pushing it, we will have to double check that this message isn’t a duplicate, so we would check to make sure it doesn’t have the exact same values as the previous message decoded.

Updating the board state after the opponent moves

For this next section, we should have a function such as SendBoardState(Board board) that would be called by the game logic team. This function should take the board from the argument and encode it into a byte format to be sent over the network and place it within a local variable to be sent off when the connection is ready to do as such.

Another Sister function will be needed to decode this information

In addition, we should have a EndUsersTurn() function that lets the server know that this players turn is done and thus the other players turn has begun

As well as a sister function SetUsersTurn() that sets lets the user know it their turn (bad way of explaining it, but I can't think of any other way at the moment)

The Main Loop

This is the core of our networking team, as it will act as the central location that uses most of the functions we create

The main networking loop must be multithreaded as networking spends a lot of its time waiting for a packet, and while waiting no other code can be executed, thus we must have it in another thread. This means that our networking class will **extend** "`Thread`" which is java's basic class for multithreading.

To utilize the multithreading we will need a run() function within the class, and this is what will be run when OurClass.start() is called

The actual client-side loop will kinda go something like this.

1. Package/Send All data to the server
2. Wait For servers package
3. Recive package and decode data, update content with this new data

We should also have a requestGame() function that lets this loop know to send a request to the server

Before we enter the main loop of our networking code we should have a pre-loop of sorts that waits and checks for authentication and only proceed if the server responds with a positive or we cannot contact the server at as a fail safe

Structure Of the Sent Data

Feel free to recommend another method of sending our data on the network, I thought this up in the shower lol, so any improvements would be appreciated

My initial thoughts on this is for each "object/value/variable" we send over the network we should pair it with two 4 byte int's proceeding it. The first acts as a guide for how many bytes it should read for this "value" and the second as an identifier as to what is being sent over.

So it would look kind of like this

"0000 .... 0101 0000 .... 0001 1000 0101 .... 0000"

So the yellow section would inform the code about how many bits this value its about to read will be and the purple section tells it what those bits represent, eg the value 1 means its the leaderboard data or something like that

And so when we want to send data we will just append everything that needs to be sent into one packet and use this to decode it

Alternatively, we could also skip the identifying bits and just hard code lengths for each packet, but this seems like a headache

And our last option that I can think of right now is we could look for a library that has a simplified networking protocol and use that

The Server-Side Loop

The server-side loop is not our top priority, but I do think we can get it done within the time frame, It will consist of 3 loops all running on different threads at the same time

The first loop is pretty simple it will, wait for connections accept them and pass them onto the second loop

The Second loop is our authentication loop, it will compare the given username and password with the username and pass it has on file. This loop will also be responsible for the creating new users and adding them to the file

The 3rd loop is our main loop that mirrors the main loop of the client, it should send some data regularly, such as the leaderboard information, and also wait and listen to the client for a game request, from there it would call the matchmaking system to see which of the players it should match up together, after that it should add both of these to some sort of list. The Server should then relay the other players data to each other using this list, appending it to the regular data being sent. Do this till someone wins the game and change the players stats and close the connection